

Mutual Exclusion with Busy Waiting

Part 2: Peterson's solution, TSL, and XCHG

Focus of This Note

This note continues busy-waiting mutual exclusion by studying one software solution, **Peterson's solution**, and two hardware-assisted solutions, **TSL** and **XCHG**.

1. From Strict Alternation to Better Ideas

Problem to Solve

Strict alternation prevents two processes from entering their critical regions at the same time, but it can block a process even when the other process is outside its critical region.

Better Strategy

A better software solution combines two ideas:

- a process announces that it is interested in entering the critical region,
- a shared `turn` variable breaks ties when both processes are interested.

2. Peterson's Solution

Peterson's Algorithm

Peterson's solution is a software algorithm for mutual exclusion between two processes. It avoids strict alternation by using both an interest array and a turn variable.

The shared data are:

- `turn`: indicates which process should wait if both want to enter.
- `interested[N]`: records whether each process wants to enter its critical region.

```
#define FALSE 0
#define TRUE 1
#define N 2

int turn;
int interested[N]; /* all values initially FALSE */

void enter_region(int process)
{
    int other;

    other = 1 - process;
    interested[process] = TRUE;
    turn = process;
```

```

while (turn == process && interested[other] == TRUE)
{
    /*loop tasks*/
}

void leave_region(int process)
{
    interested[process] = FALSE;
}

```

How to Use It

Before entering the critical region, a process calls `enter_region(process)`. After leaving the critical region, it calls `leave_region(process)`.

3. How Peterson's Solution Works

Situation	Result
Only process 0 is interested	Process 0 enters immediately because process 1 is not interested.
Only process 1 is interested	Process 1 enters immediately because process 0 is not interested.
Both become interested almost together	Both set their interest flags, but the final write to <code>turn</code> decides who waits.
One process leaves	It sets its interest flag to <code>FALSE</code> , allowing the other waiting process to continue.

Tie-Breaking Example

Suppose both processes call `enter_region` at about the same time. If process 1 stores into `turn` last, then `turn = 1`. Process 1 waits, while process 0 enters the critical region.

Why This Is Better Than Strict Alternation

If the other process is not interested, a process can enter immediately. It does not have to wait merely because it is the other process's turn.

Modern Systems Note

Peterson's solution is a classic conceptual algorithm. Real modern systems usually need atomic variables or memory barriers because compilers and CPUs may reorder memory operations.

4. Peterson's Solution and the Four Conditions

Condition	How Peterson's Solution Handles It
Mutual exclusion	Both processes cannot pass the waiting test at the same time when both are interested.
No speed assumptions	The algorithm does not rely on one process being faster than the other.
No outside blocking	A process that is not interested has <code>interested[process] = FALSE</code> , so it does not block the other.
No endless waiting	The waiting process can enter after the other process leaves and clears its interest flag.

5. The TSL Instruction

TSL

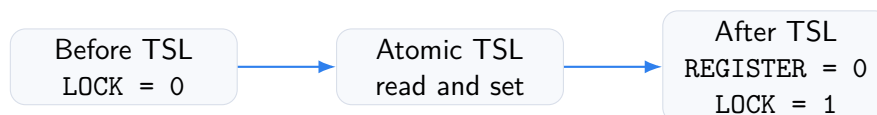
TSL means **Test and Set Lock**. It is a hardware instruction that reads a lock value and sets the lock in one indivisible operation.

The instruction is usually written like this:

```
TSL REGISTER, LOCK
```

It does two things atomically:

1. copy the old value of `LOCK` into `REGISTER`,
2. store a nonzero value into `LOCK`.



Why Atomicity Matters

No other processor can access the lock word between the read and the write. This removes the race that broke the simple lock-variable solution.

6. Using TSL for Mutual Exclusion

```
enter_region:
    TSL REGISTER, LOCK ; copy LOCK to REGISTER and set LOCK to 1
    CMP REGISTER, #0 ; was LOCK zero?
    JNE enter_region ; if not zero, lock was already set
    RET ; critical region entered

leave_region:
    MOVE LOCK, #0 ; release the lock
    RET
```

How the Code Works

If the old value of `LOCK` was 0, the process has acquired the lock and enters the critical region. If the old value was not 0, another process already has the lock, so the process loops and tries again.

Still Busy Waiting

TSL makes lock acquisition correct, but the waiting process still spins in a loop. It is safe mutual exclusion, but it can waste CPU time.

7. TSL vs. Disabling Interrupts

Technique	What It Controls
Disabling interrupts	Stops interrupts only on the CPU that disabled them. It does not stop other CPUs.
TSL / atomic test-and-set	Locks the memory operation itself, so other processors cannot slip between the read and write of the lock.

Key Difference

Disabling interrupts controls scheduling on one CPU. TSL controls access to the shared memory word itself.

8. The XCHG Instruction

XCHG

XCHG atomically exchanges the contents of two locations, such as a register and a memory word. It can be used to build the same kind of spin lock as TSL.

```
enter_region:
    MOVE REGISTER, #1 ; put 1 in the register
    XCHG REGISTER, LOCK ; atomically swap REGISTER and LOCK
    CMP REGISTER, #0 ; was LOCK zero before the swap?
    JNE enter_region ; if not zero, lock was already set
    RET ; critical region entered

leave_region:
    MOVE LOCK, #0 ; release the lock
    RET
```

How XCHG Works

The process places 1 in a register, then atomically swaps it with `LOCK`. If the register receives 0, the lock was free and is now acquired. If the register receives 1, the lock was already taken and the process keeps spinning.

9. TSL and XCHG Compared

Instruction	Atomic Action	Use in Mutual Exclusion
TSL	Reads the old lock value and sets the lock to nonzero.	If the old value was 0, the caller acquired the lock.
XCHG	Swaps a register value with the lock value.	If the old lock value moved into the register was 0, the caller acquired the lock.

Low-Level Synchronization

Hardware atomic instructions such as TSL and XCHG are building blocks for low-level locks used by operating systems and runtime libraries.

10. Cooperation Requirement

Processes Must Follow the Protocol

Critical-region solutions only work if processes call `enter_region` before touching shared data and call `leave_region` after finishing. If a process ignores the protocol, mutual exclusion fails.

Correct Lock Discipline

A process should hold the lock only while it is actually using the protected shared data. Keeping the lock too long increases waiting and reduces parallelism.

11. When Busy Waiting Makes Sense

Good Fit	Poor Fit
The wait is expected to be very short.	The wait may be long or unpredictable.
The cost of sleeping and waking is higher than spinning briefly.	CPU time is valuable and should not be burned in a loop.
Low-level kernel or multiprocessor synchronization.	General user-level waiting for I/O or long critical regions.

12. Quick Review

Key Points

- Peterson's solution uses both `interested[]` and `turn`.
- If only one process is interested, it enters immediately.
- If both are interested, `turn` decides which process waits.
- TSL atomically reads the old lock value and sets the lock.
- XCHG atomically swaps a register and the lock variable.
- Hardware atomic instructions fix the race in simple lock variables.
- TSL and XCHG still use busy waiting, so they can waste CPU time.
- Critical-region methods require all processes to cooperate with the entry and exit protocol.